

Projeto CRUD em PHP (com Banco de Dados)

Um CRUD representa um conjunto de operações básicas que consistem em **Create** (criar), **Read** (ler), **Update** (atualizar) e **Delete** (apagar). Para realizar este projeto com sucesso, são necessários elementos como a conexão com banco de dados, criação de páginas, lógica em PHP, validação de dados e exibição de resultados.

Etapa 1: Preparação do Banco de dados

Vamos detalhar a primeira parte do projeto, que é fundamental para tudo funcionar. Se o banco de dados não for criado corretamente no **phpMyAdmin**, o código PHP não terá onde salvar as informações. Para este passo a passo, assumiremos que você está usando um servidor local (como **XAMPP**, **WAMP** ou **MAMP**), que já vem com o PHP, MySQL e o phpMyAdmin embutidos.

Passo 1: Iniciando o Servidor Local

1. Abra o painel de controle do seu servidor local (ex: XAMPP Control Panel).
2. Inicie os serviços **Apache** (o servidor web) e **MySQL** (o banco de dados). Ambos devem ficar com o status "Verde" ou "Rodando".

Passo 2: Acessando o phpMyAdmin

1. Abra o seu navegador de internet (Chrome, Edge, Firefox, etc.).
2. Na barra de endereços, digite: `http://localhost/phpmyadmin` e aperte Enter.
3. Isso abrirá a interface visual do phpMyAdmin, que é um gerenciador de banco de dados MySQL. *(Em servidores locais padrão, não é exigido usuário e senha para entrar; você já cai direto no painel logado como administrador).*

Passo 3: Criando o Banco de Dados

1. No menu lateral esquerdo do phpMyAdmin, clique na primeira opção chamada **"Novo"** (ou "New").
2. No campo de texto escrito **"Nome do banco de dados"**, você deve digitar exatamente o nome que usou no seu código `conexao.php`. Portanto, digite: `crudphp`.
3. Ao lado do nome, há um menu suspenso chamado *Collation* (Agrupamento). É recomendável procurar e selecionar `utf8mb4_general_ci` ou `utf8mb4_unicode_ci`. Isso garante que o banco de dados aceitará perfeitamente acentuação (ç, á, ã) dos nomes inseridos.
4. Clique no botão **"Criar"**.

Passo 4: Criando a Tabela `cadastro`

Assim que você criar o banco de dados, o phpMyAdmin o selecionará automaticamente e mostrará uma tela para criar uma nova tabela.

1. No campo **"Nome da tabela"**, digite: `cadastro` (é este nome que seu arquivo `cadaststrar.php` está buscando no comando `INSERT INTO cadastro...`).
2. No campo **"Número de colunas"**, digite: `4` (já que vamos guardar ID, Nome, Sobrenome e Data de Nascimento).
3. Clique em **"Criar"** (ou "Executar").

Passo 5: Configurando as Colunas da Tabela

Agora você verá uma tela com 4 linhas em branco para configurar as propriedades de cada coluna. Preencha exatamente da seguinte forma:

Linha 1 (O Identificador Único):

- Nome: `id`
- Tipo: `INT` (Inteiro)

- Índice (Index): Clique no menu suspenso e escolha **PRIMARY** (Chave Primária). Uma janelinha pode abrir para confirmar, basta clicar em Executar.
- A.I. (Auto Increment): Marque esta caixinha de seleção. Isso é crucial, pois diz ao banco para numerar os cadastros automaticamente (1, 2, 3...) sem que você precise enviar o ID no PHP.

Linha 2 (O Nome):

- Nome: **nome**
- Tipo: **VARCHAR** (Texto variável)
- Tamanho/Valores: **255** (Limite máximo de caracteres)

Linha 3 (O Sobrenome):

- Nome: **sobrenome**
- Tipo: **VARCHAR**
- Tamanho/Valores: **255**

Linha 4 (A Data de Nascimento):

- Nome: **datanasc**
- Tipo: **DATE** (Formato específico para datas: AAAA-MM-DD)

Após preencher as 4 colunas, role a tela até o final e clique no botão **"Guardar"** (ou "Salvar"). Pronto! A estrutura do seu banco de dados está perfeita e pronta para receber informações.

Passo 6: Entendendo a Conexão no PHP (**conexao.php**)

Agora que o banco existe no phpMyAdmin, o seu código já consegue "enxergá-lo". Olhando o seu arquivo **conexao.php**:

```
PHP
$pdo = new PDO('mysql:host=localhost;dbname=crudphp', 'root', "");
```

Aqui está a explicação detalhada do que cada pedaço dessa linha faz:

- **new PDO(...)**: Cria uma nova conexão usando a tecnologia PDO (PHP Data Objects), que é a forma mais segura e moderna de conectar bancos de dados no PHP.
- **mysql:host=localhost**: Diz ao PHP qual é o tipo de banco (MySQL) e onde ele está hospedado (localhost, ou seja, no mesmo computador onde o PHP está rodando).
- **dbname=crudphp**: Informa ao PHP o nome exato do banco de dados que acabamos de criar no Passo 3.
- **'root'**: Este é o nome do **usuário** administrador padrão do MySQL no XAMPP/WAMP.
- **" (aspas vazias)**: Esta é a **senha**. Por padrão, em ambiente local, o usuário root não possui senha, por isso deixamos vazio. Se você estivesse hospedando isso em um servidor online real, haveria uma senha forte aqui.

Etapa 2: Configuração Detalhada da Conexão (**conexao.php**)

Este arquivo funciona como a "ponte" de comunicação indispensável do seu projeto. Sem ele, as páginas PHP (**index.php**, **cadastrear.php**, etc.) funcionariam de forma isolada, sendo incapazes de salvar ou ler qualquer informação armazenada no banco de dados que criamos no phpMyAdmin.

Para criar essa ponte de maneira limpa e profissional, o código utiliza o **PDO (PHP Data Objects)**. O PDO é uma extensão do PHP que define uma interface leve e consistente para acessar bancos de dados, oferecendo camadas extras de segurança (como proteção contra ataques de *SQL Injection*).

Abaixo está o desmembramento absoluto da única — mas poderosa — linha contida no seu arquivo **conexao.php**:

```
PHP
<?php
```

```
$pdo = new PDO('mysql:host=localhost;dbname=crudphp', 'root', '');
```

1. Abertura da Tag PHP (<?php)

- Todo arquivo que processa instruções lógicas de PHP precisa iniciar com a tag `<?php`. Como este é um arquivo de configuração pura (ele não renderiza HTML, não tem textos estruturados nem estilos), não é necessário fechar a tag com `?>` no final. Manter sem o fechamento evita que espaços em branco acidentais quebrem os redirecionamentos de página (`header('Location: ...')`) no futuro.

2. Criação da Variável de Conexão (\$pdo)

- O símbolo `$` indica a criação de uma variável. O nome escolhido foi `pdo`. Esta variável passará a carregar o objeto da conexão ativa. Sempre que qualquer outro arquivo do seu sistema (como o `cadastrar.php`) precisar conversar com o banco, ele chamará essa variável `$pdo` para dar as ordens.

3. Instanciação do Objeto (= new PDO(...))

- O termo `new` serve para instanciar (criar) um novo objeto baseado em uma classe. Estamos dizendo ao PHP: *"Instancie um novo mecanismo de conexão usando as regras da classe PDO"*. Dentro dos parênteses, passamos os **três parâmetros fundamentais** exigidos pelo PDO:

Anatomia dos Parâmetros do PDO

O construtor do PDO recebe três argumentos em formato de texto (strings), separados por vírgula: `new PDO('DSN', 'Usuário', 'Senha')`.

Parâmetro 1: O DSN (Data Source Name) → `'mysql:host=localhost;dbname=crudphp'`

O DSN é uma string de texto que diz ao PHP qual é o tipo de banco de dados, onde ele está e qual banco específico deve ser aberto. Ela é dividida em três partes:

- **mysql:** Define o *driver* de conexão. Avisa ao PHP que o banco de dados de destino é do tipo MySQL/MariaDB (o phpMyAdmin gerencia esse tipo).
- **host=localhost** Define o endereço do servidor. Como você está desenvolvendo na sua própria máquina de forma local utilizando um pacote como o XAMPP ou WAMP, o servidor de banco de dados está rodando no endereço local, identificado pela palavra-chave `localhost`.
- **dbname=crudphp** Especifica o nome exato do banco de dados (esquema) que foi criado previamente dentro do phpMyAdmin. O PHP tentará se conectar a essa pasta virtual. Se houver qualquer erro de digitação aqui (como escrever `crud_php`), a conexão falhará com um erro de "banco desconhecido".

Parâmetro 2: O Usuário → `'root'`

- É o segundo argumento inserido nos parênteses. Define as credenciais de acesso. Ambientes de desenvolvimento locais (XAMPP, WAMP, MAMP) vêm configurados de fábrica com um usuário administrador supremo padrão chamado `root`. Esse usuário tem permissões totais para criar, ler, modificar e deletar qualquer dado.

Parâmetro 3: A Senha → `"`

- É o terceiro argumento. Por padrão, em servidores locais voltados para aprendizado ou testes locais, o usuário `root` vem configurado **sem nenhuma senha**. No PHP, representamos a ausência de caracteres abrindo e fechando aspas simples imediatamente `"` (ou aspas duplas `""`).
- *Nota de segurança:* Caso você envie esse projeto para um servidor de hospedagem real na internet (produção), você obrigatoriamente definirá uma senha complexa aqui por motivos de segurança.

Como este arquivo é reaproveitado no sistema?

Para não ter que digitar essa linha de conexão em todas as páginas do sistema, o PHP permite importar arquivos de configuração. No início do seu arquivo `cadastrar.php` e do seu `index.php`, você notará a seguinte linha:

PHP

```
require('conexao.php');
```

O comando `require` faz com que o PHP "copie e cole" todo o conteúdo de `conexao.php` para dentro da outra página no exato momento da execução. Dessa forma, a variável `$pdo` fica disponível instantaneamente para preparar e executar os comandos SQL que inserem e listam os dados na tela.

Etapa 3: Desenvolvimento do Formulário (`index.php`)

Agora vamos mergulhar na **Etapa 3**, que trata da interface com o usuário: o arquivo `index.php`. Enquanto o arquivo de conexão trabalha nos bastidores, o `index.php` é a "vitrine" do seu sistema. É a tela que o usuário final verá no navegador. Para que os dados cheguem até o banco de dados, precisamos de uma porta de entrada, e essa porta é o **formulário HTML**.

Analisando o código do seu arquivo `index.php`, temos o seguinte bloco responsável por essa etapa: HTML

```
<form action="cadastrar.php" method="post">
  <input class="form-control" type="text" name="nome" placeholder="Digite seu nome">
  <input class="form-control" type="text" name="sobrenome" placeholder="Digite seu sobrenome">
  <input class="form-control" type="date" name="datanasc" placeholder="Digite sua data de
nascimento">
  <input type="submit" value="Cadastrar" class="btn btn-primary">
</form>
```

Vamos desmembrar cada elemento e entender o papel de cada atributo detalhadamente:

1. A Tag `<form>`: O Envelope de Dados

A tag `<form>` funciona como um "envelope" que agrupa os dados que o usuário vai digitar. Tudo o que estiver dentro dela será empacotado e enviado de uma só vez quando o botão for clicado.

- **`action="cadastrar.php"`**: Este é o endereço de destino. Diz ao navegador: *"Quando o usuário clicar em enviar, pegue todos os dados deste formulário e entregue-os para o arquivo `cadastrar.php` processar"*.
- **`method="post"`**: Define a forma como os dados serão transportados. O método `POST` envia as informações de forma "invisível" no corpo da requisição do navegador. Diferente do método `GET` (que mostra os dados na URL do navegador), o `POST` é muito mais seguro e adequado para cadastros, pois não expõe os dados sensíveis e permite o envio de um volume maior de informações.

2. Os Campos de Texto (`<input type="text">`)

Temos dois campos para capturar o nome e o sobrenome do usuário.

- **`type="text"`**: Diz ao navegador para renderizar uma caixa de texto simples.
- **`name="nome"` e `name="sobrenome"`**: **Este é o atributo mais importante para o PHP!** O atributo `name` funciona como a "etiqueta" daquele dado. Quando a informação chegar no arquivo `cadastrar.php`, o PHP vai procurar exatamente por essas etiquetas para resgatar o que foi digitado (através do comando `$_POST['nome']`, por exemplo).
- **`placeholder="..."`**: É aquele texto clarinho que fica dentro do campo vazio dando uma dica ao usuário (ex: "Digite seu nome"). Ele some assim que a pessoa começa a digitar.
- **`class="form-control"`**: Esta é uma classe do framework CSS **Bootstrap**. Ela não tem função no PHP, mas serve para deixar o visual do campo bonito, arredondado e ocupando 100% da largura disponível na tela.

3. O Campo de Data (`<input type="date">`)

- **`type="date"`**: Uma funcionalidade excelente do HTML5. Ao definir o tipo como `date`, navegadores modernos (como Chrome, Firefox, Edge) exibem automaticamente um mini-calendário (date picker) para o usuário escolher a data. Isso evita que o usuário digite a data em um formato incorreto (como "10/05/90" em vez do formato de banco de dados "1990-05-10").

- **name="datanasc"**: A etiqueta que identificará essa data lá no processamento do PHP. Lembre-se: criamos uma coluna com esse mesmo nome no banco de dados.

4. O Botão de Envio (<input type="submit">)

Por fim, precisamos de um gatilho para disparar o processo.

- **type="submit"**: Transforma este input em um botão de envio. A função exclusiva do tipo "submit" é procurar a tag <form> onde ele está inserido, empacotar todos os inputs que têm um name, e enviá-los para a URL definida no action.
- **value="Cadastrar"**: É o texto que aparecerá escrito no meio do botão.
- **class="btn btn-primary"**: Mais classes do Bootstrap. btn avisa que o elemento deve ter formato de botão, e btn-primary aplica a cor primária padrão do framework (geralmente azul).

Resumindo: Quando o usuário preenche essas caixas e clica em "Cadastrar", o navegador cria um pacote oculto (POST) contendo variáveis como nome="João", sobrenome="Silva", datanasc="1990-01-01" e as arremessa diretamente para o arquivo cadastrar.php, que ficará encarregado de abrir esse pacote e guardá-lo no banco de dados.

Etapa 4: Lógica de Processamento (cadastrar.php)

Chegamos à **Etapa 4**, o "cérebro" da operação de criação (Create). O arquivo cadastrar.php não possui interface visual (HTML); o trabalho dele é puramente nos bastidores. Ele atua como um mensageiro que pega o pacote de dados enviado pelo formulário, verifica as informações, abre a porta do banco de dados e guarda tudo lá dentro de forma segura.

Vamos desmembrar o código do seu arquivo cadastrar.php linha por linha para entender essa lógica perfeitamente:

1. Chamando a Conexão

PHP

```
require('conexao.php');
```

Antes de fazer qualquer coisa, o script precisa saber como acessar o banco de dados. O require importa o arquivo que configuramos no **Passo 2**, disponibilizando a variável \$pdo (que contém a conexão ativa) para ser usada aqui.

2. Captura e Filtragem dos Dados

PHP

```
$nome = filter_input(type:INPUT_POST, var_name:'nome');  
$sobrenome = filter_input(type:INPUT_POST, var_name:'sobrenome');  
$datanasc = filter_input(type:INPUT_POST, var_name:'datanasc');
```

Quando o formulário (do index.php) é enviado, os dados chegam através de um método chamado POST. Para capturar essas informações, utilizamos a função embutida do PHP filter_input().

- **Por que usar filter_input?** Em vez de simplesmente pegar os dados de forma bruta (usando \$_POST['nome']), o filter_input é uma prática recomendada de segurança. Ele verifica se a variável realmente existe, evitando erros na tela caso o usuário tente acessar a página diretamente sem enviar o formulário.
- Os dados capturados são então guardados em variáveis locais do PHP (\$nome, \$sobrenome, \$datanasc) para facilitar o manuseio.

3. O Bloco Try-Catch (Tentativa e Erro)

PHP


```
try {
    // Código de inserção no banco...
} catch (PDOException $e) {
    // Tratamento de erro...
}
```

Mexer com banco de dados é uma operação "arriscada": o servidor pode cair, a tabela pode não existir, um dado pode estar no formato errado. O bloco `try-catch` é uma rede de segurança.

- **try { ... }:** Diz ao PHP: *"Tente executar os comandos aqui dentro"*. Se tudo der certo, ele segue o fluxo normalmente.
- **catch (...) { ... }:** Se *qualquer coisa* der errado dentro do `try`, o PHP para imediatamente e "pula" para o `catch`. Isso evita que seu sistema trave de forma feia ou exponha dados sensíveis aos usuários.

4. A Query SQL e a Prevenção contra Invasões (SQL Injection)

Dentro do bloco `try`, temos a preparação da ordem que será dada ao banco de dados:

PHP

```
$sql = "INSERT INTO cadastro (nome, sobrenome, datanasc) VALUES (:nome, :sobrenome, :datanasc)";
$stmt = $pdo->prepare($sql);
```

- **A Query (\$sql):** Estamos escrevendo o comando SQL `INSERT INTO`. Dizemos para inserir na tabela `cadastro`, especificamente nas colunas `nome`, `sobrenome` e `datanasc`.
- **Os Marcadores (:nome):** Em vez de colocar as variáveis diretas na query (o que abriria uma brecha gravíssima de segurança chamada *SQL Injection*, onde hackers podem destruir seu banco), usamos **marcadores virtuais** iniciados por dois pontos (como `:nome`).
- **\$pdo->prepare(\$sql):** Dizemos ao banco: *"Prepare-se, vou mandar essa estrutura de texto, mas os valores reais enviarei em seguida"*. Isso cria um objeto chamado `$statement` (declaração).

5. Atribuindo os Valores (bindValue) e Executando

PHP

```
$statement->bindValue(':nome', $nome);
$stmt->bindValue(':sobrenome', $sobrenome);
$stmt->bindValue(':datanasc', $datanasc);
$stmt->execute();
```

- **bindValue():** É aqui que ligamos os pontos. Nós informamos ao PDO qual é o valor real de cada marcador. Exemplo: *"Lembra do marcador :nome? Substitua ele pela variável \$nome que capturamos lá em cima"*. Como o banco já pré-compilou a estrutura da query no `prepare()`, ele agora trata essa variável **estritamente como um texto comum**, neutralizando qualquer código malicioso que um hacker pudesse ter digitado no formulário.
- **execute():** Este é o gatilho final. Ele manda o comando completo e montado para o banco de dados executar. É neste exato milissegundo que a nova linha é criada na sua tabela do phpMyAdmin.

6. Redirecionamento de Volta

PHP

```
header('Location: index.php');
```

Se o `execute()` funcionou sem erros, a linha de baixo entra em ação. A função `header('Location: ...')` emite uma ordem ao navegador do usuário: *"O cadastro foi feito! Volte instantaneamente para a página index.php"*. Sem isso, o usuário ficaria preso em uma tela toda branca após clicar no botão de enviar.

7. O Tratamento de Falhas (O Catch)

PHP

```
} catch (PDOException $e) {  
    echo "Erro ao cadastrar usuário: " . $e->getMessage();  
    exit();  
}
```

Se algo der errado (por exemplo, você esqueceu de iniciar o servidor MySQL no XAMPP), o bloco `try` falha e o `catch` assume.

- A variável `$e` guarda todos os detalhes técnicos do erro.
- Usamos `echo` para imprimir uma mensagem amigável juntamente com a descrição do erro (`$e->getMessage()`), facilitando a vida do programador para descobrir o que quebrou.
- A função `exit()` obriga o PHP a encerrar o carregamento da página ali mesmo, garantindo que nenhum outro código accidental seja rodado após o erro.

Análise da Etapa de Criação (Create)

Até este momento, o material fornecido construiu uma base robusta para o desenvolvimento de um sistema **CRUD** (Create, Read, Update, Delete) em PHP. A arquitetura estruturada nesta aula reflete excelentes práticas de programação voltadas para a segurança e organização do código.

- **Modelagem do Banco de Dados:** A preparação inicial no phpMyAdmin, com a criação do banco `crudphp` e da tabela `cadastro`, foi bem fundamentada. A escolha de tipos de dados adequados (como `DATE` para a coluna `datanasc` e `VARCHAR 255` para os textos) garante a integridade da informação logo em sua origem.
- **Conexão e Segurança:** A utilização da extensão PDO (PHP Data Objects) atua como um pilar central de segurança. O PDO padroniza o acesso e cria uma barreira nativa contra ataques de SQL Injection, que são invasões que se aproveitam de brechas em formulários.
- **Interface Front-end:** O arquivo `index.php` foi estabelecido como a interface do usuário, utilizando um formulário com o método `POST`. O método `POST` transporta os dados de forma invisível no corpo da requisição, sendo a escolha ideal e segura para cadastros.
- **Processamento Back-end:** O arquivo `cadastar.php` demonstra maturidade no tratamento de dados. A adoção da função `filter_input` assegura que as variáveis sejam tratadas corretamente e evita erros de acesso direto. O uso de *Prepared Statements* (declarações preparadas), separando a estrutura do SQL (`$pdo->prepare()`) da injeção de valores (`bindValue()`), força o banco de dados a tratar as entradas estritamente como texto inofensivo. O tratamento de erros com o bloco `try-catch` fecha o pacote garantindo que falhas técnicas sejam contidas.

Conclusão da Aula de Criação

A etapa de "Create" foi concluída de maneira muito eficiente. Estabelecemos um fluxo unidirecional onde o usuário insere informações em um formulário visualmente limpo (com classes do Bootstrap como `form-control`), envia um pacote oculto para o servidor web, e o script PHP encarrega-se de inserir esses dados no banco de forma validada e segura. O toque final com a função de redirecionamento `header('Location: index.php')` garante uma experiência de uso contínua e sem travamentos de tela branca.

Próximos Passos: Focando no READ (Leitura e Exibição de Dados)

Agora que nosso sistema tem a capacidade de persistir (salvar) informações no banco de dados, a progressão natural é extrair essas informações para devolvê-las à interface visual. Na nossa próxima etapa, o foco será totalmente na operação **Read**.

O que abordaremos na próxima etapa?

1. Consulta de Retorno (A Query SELECT):

- Iniciaremos modificando o topo do arquivo `index.php`, adicionando uma lógica de comunicação que rodará antes de a tela carregar.
- O PHP enviará o comando `SELECT * FROM cadastro` ao banco, instruindo-o a devolver todas as linhas salvas.

2. Organizando as Respostas (O Método Fetch All):

- Utilizaremos o comando estrutural `$statement->fetchAll(PDO::FETCH_ASSOC)`. Este método coleta a resposta bruta do MySQL e a converte em um *array associativo* (uma matriz) que o PHP consegue manipular facilmente.

3. Construção da Tabela HTML Dinâmica:

- A grande mágica ocorrerá no front-end. Em vez de digitarmos os dados fixos no código HTML, utilizaremos um laço de repetição chamado `foreach` (que significa "para cada").
- O PHP irá percorrer a nossa matriz de usuários e, para cada registro encontrado, gerará dinamicamente uma nova linha (`<tr>`) dentro de uma tabela organizada pelo Bootstrap (`table-striped`) exibindo colunas como "Nome" e futuramente "Opções".

4. Teste de Integração (Criar e Ler em Tempo Real):

- Realizaremos um teste completo do nosso fluxo.
- Preencheremos o formulário de cadastro que já criamos e, ao clicar no botão "Cadastrar", o sistema processará os dados, fará o redirecionamento imediato, e você verá a nova informação surgir instantaneamente e de forma dinâmica na nossa tabela HTML.